SCRIPT-CONTROLLED ACL

RESTRICT RECORD ACCESS BASED ON FIELD VALUE

PROJECT TITLE : Script-Controlled ACL – Restrict Record Access Based on Field Value

TEAM ID : SWTID-2026-8467

TEAM SIZE : 5

TEAM LEADER : PRIYADHARSHINI N

-202400661@sigc.edu

TEAM MEMBERS : RAMYA V

-202400660@sigc.edu

LAVANYA E

-202400450@sigc.edu

DEEPA SRI AP

-202400171@sigc.edu

JEGATHEESHWARI A

-202400357@sigc.edu

DEMO VEDIO LINK:
https://drive.google.com/file/d/1keU6IcLuaXiLA0Nfc7CeKoZg0l7L8fqv/view?usp=drivesdk

PROJECT OVERVIEW:

The main purpose of this project is to provide record-level security in ServiceNow. Access to records is controlled based on the user's role and the Branch field value.

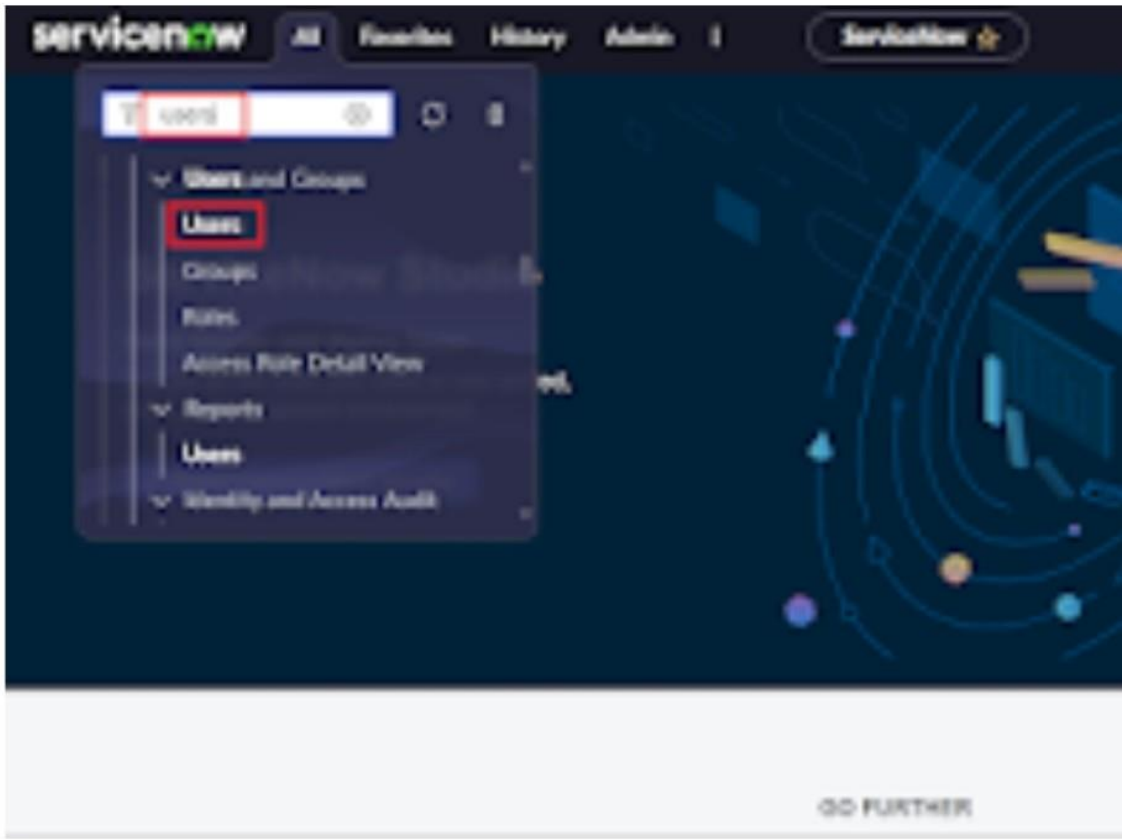
For this demonstration, four custom roles are used: bb1, bb2, bb3 and bb4. Each role has a different permission.

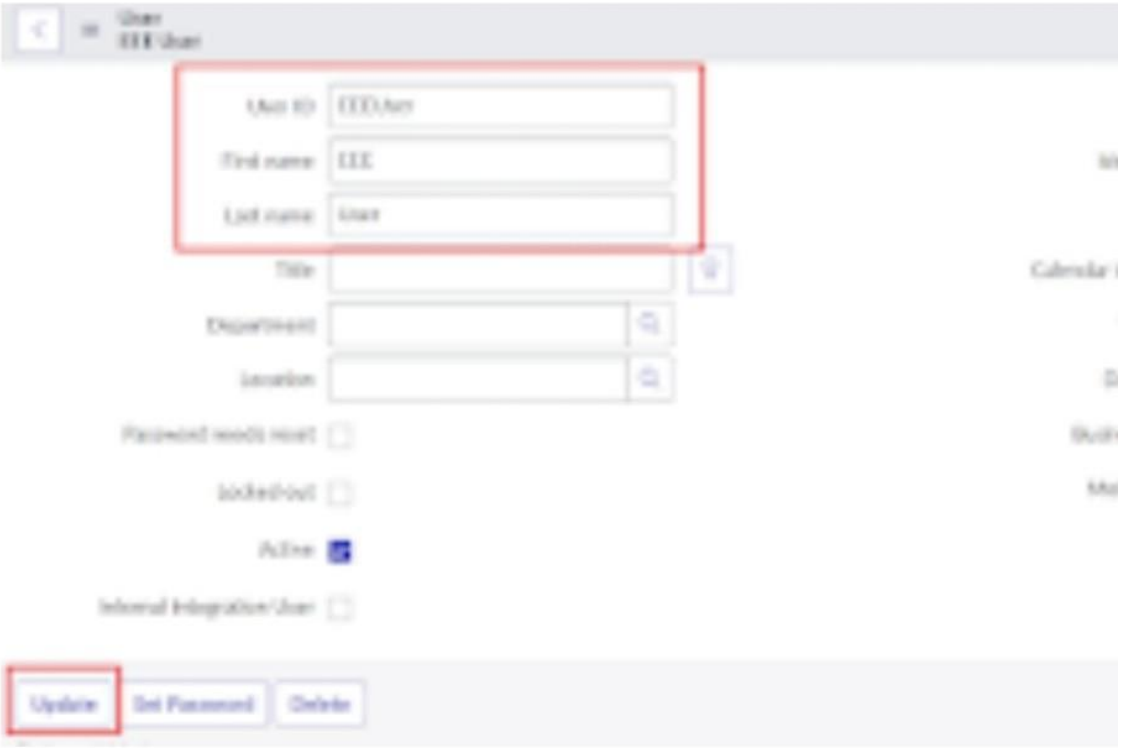
STEP 1 – USER CREATION

First, I logged into the ServiceNow instance with admin access.

Navigation: User Administration → Users

I created a test user called EEE User. After creating the user, I created four custom roles: bb1, bb2, bb3 and bb4.


servicenow
All
Favorites
History
Admin
Users and Groups
Users
Groups
Roles
Access Role Detail View
Reports
Users
Identity and Access Audit
GO FURTHER


User
First name
Last name
Title
Department
Location
Password needs reset
Locked out
Active
Internal Integration User
Update
Set Password
Delete

STEP 2 – TABLE CREATION

Next, I created a custom table called Institution Details.

Table name: u_institution_details

This table contains student-related information.

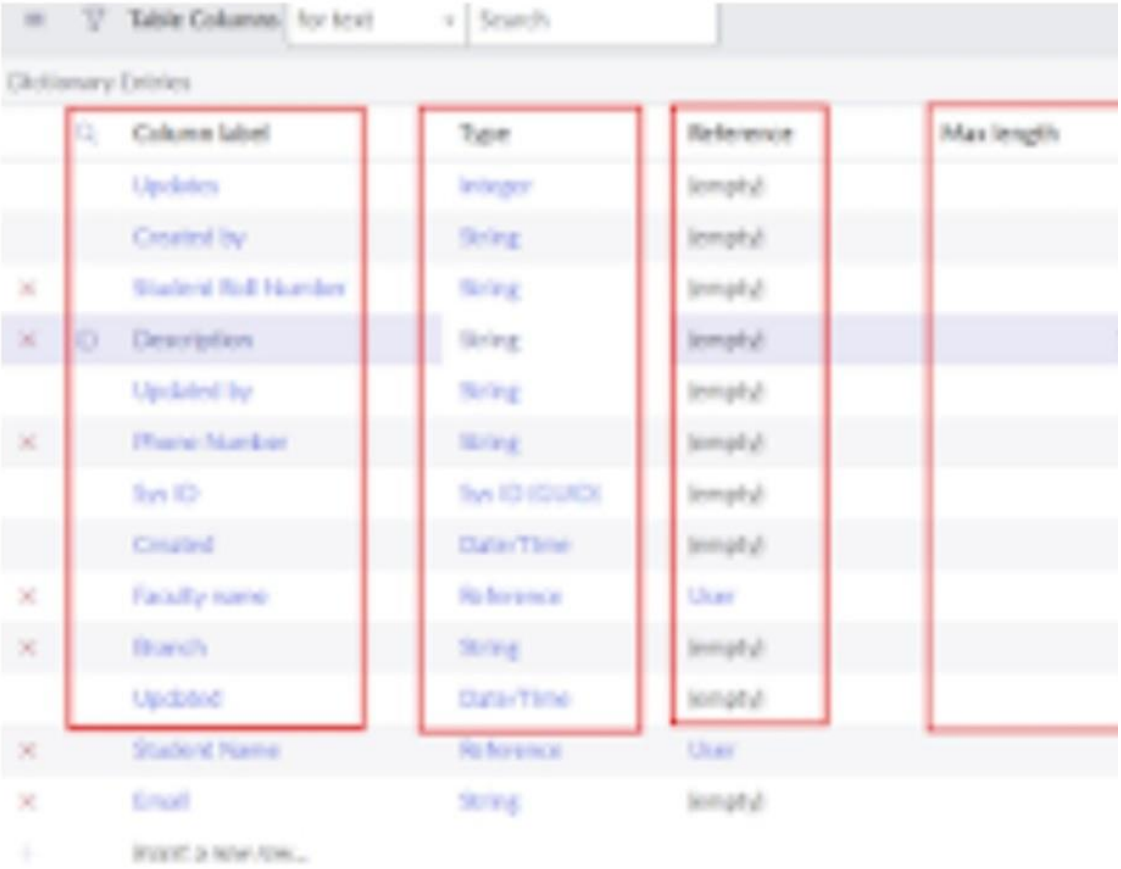
Important fields:

- Student Roll Number
- Student Name
- Faculty Name
- Branch
- Email
- Phone Number
- Description

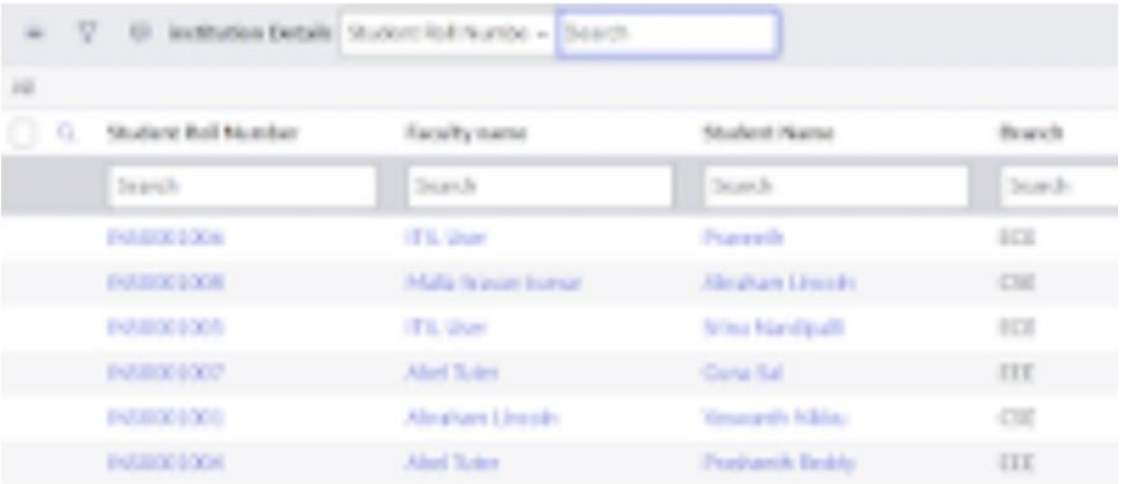
The Branch field contains values such as ECE, EEE and CSE.

Then I created multiple student records with different branch values. This is important because the ACL uses the Branch field to control access.

Screenshot – STEP 2 – TABLE CREATION



6. Create multiple records in the **Student Records** table with different branch values (ECE, EEE, CSE).



STEP 3 – READ ACL

Now I created the first ACL, which is the Read ACL.

Navigation: System Security → Access Control

I created a new record ACL. The operation is Read and the table is u_institution_details.

I added the role bb1.

I also added the data condition: Branch is EEE.

Then I used a script to control the access.

Basic logic of the script:

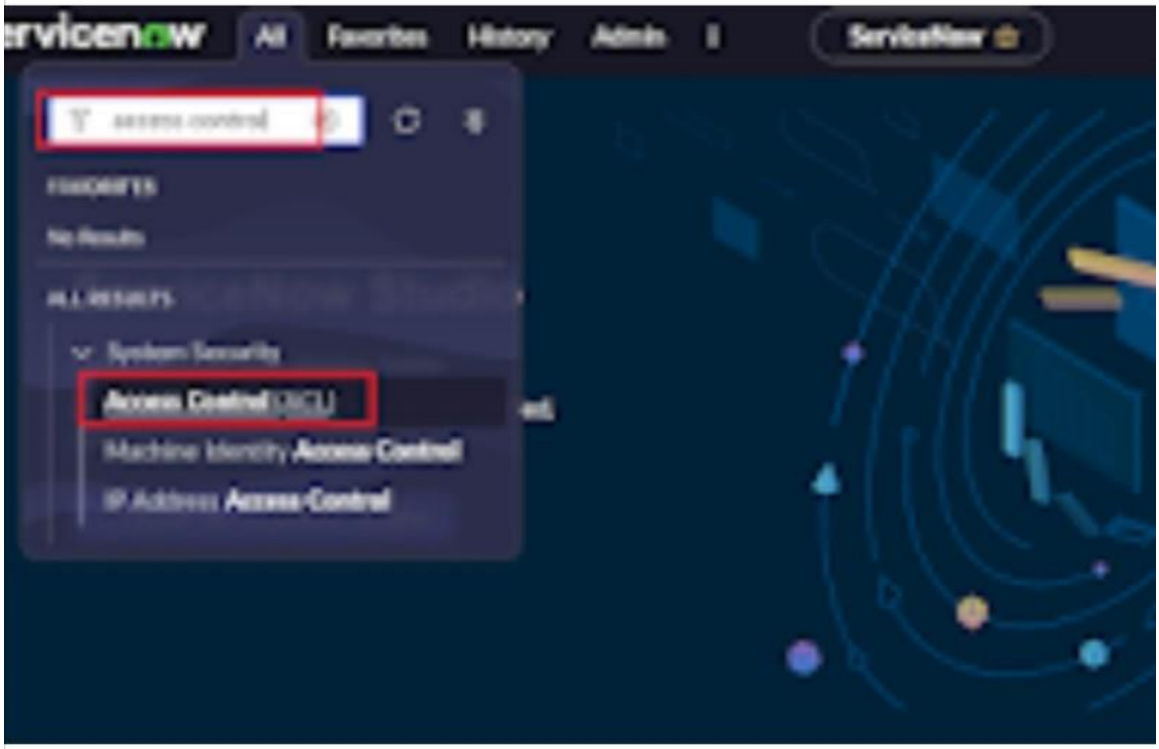
- If the user is an admin, allow access.
- If the user has the bb1 role, allow access according to the ACL condition.
- For other users, access is denied.

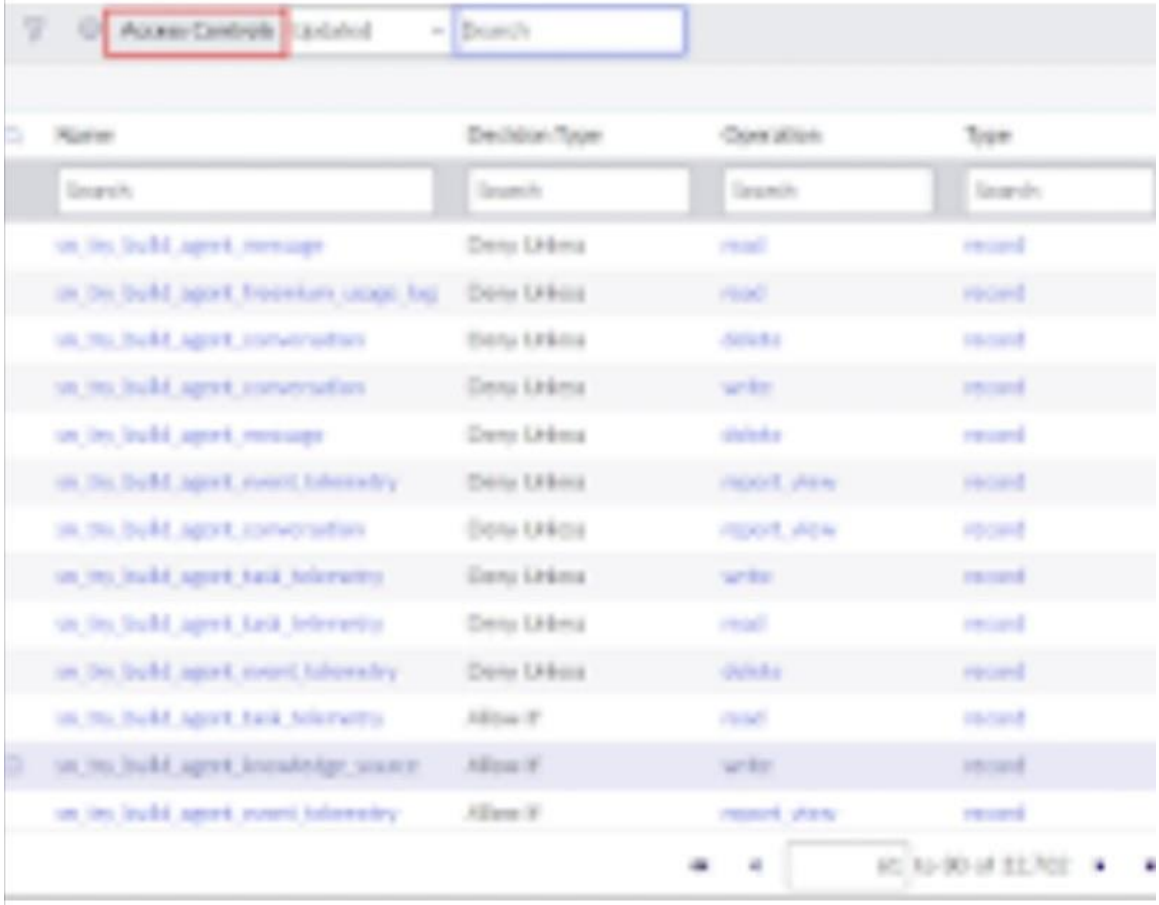
After saving the ACL, I tested it by impersonating the EEE user.

Result: A user with the bb1 role can view only EEE branch records. Users without the required permission cannot view the records. The admin user can still view all records.

This demonstrates record-level read security.

Screenshot – STEP 3 – READ ACL


All
Favorites
History
Admin
ServiceNow
access control
FAVORITES
No results
ALL RESULTS
System Security
Access Control [sec_ac]
Machine Identity Access Control
IP Address Access Control


Access Controls
Name
Decision Type
Operation
Type
Search

Access Control

STEP 4 – CREATE ACL

Next, I created the Create ACL.

The operation is Create and the table is u_institution_details.

For this ACL, I assigned the role bb2.

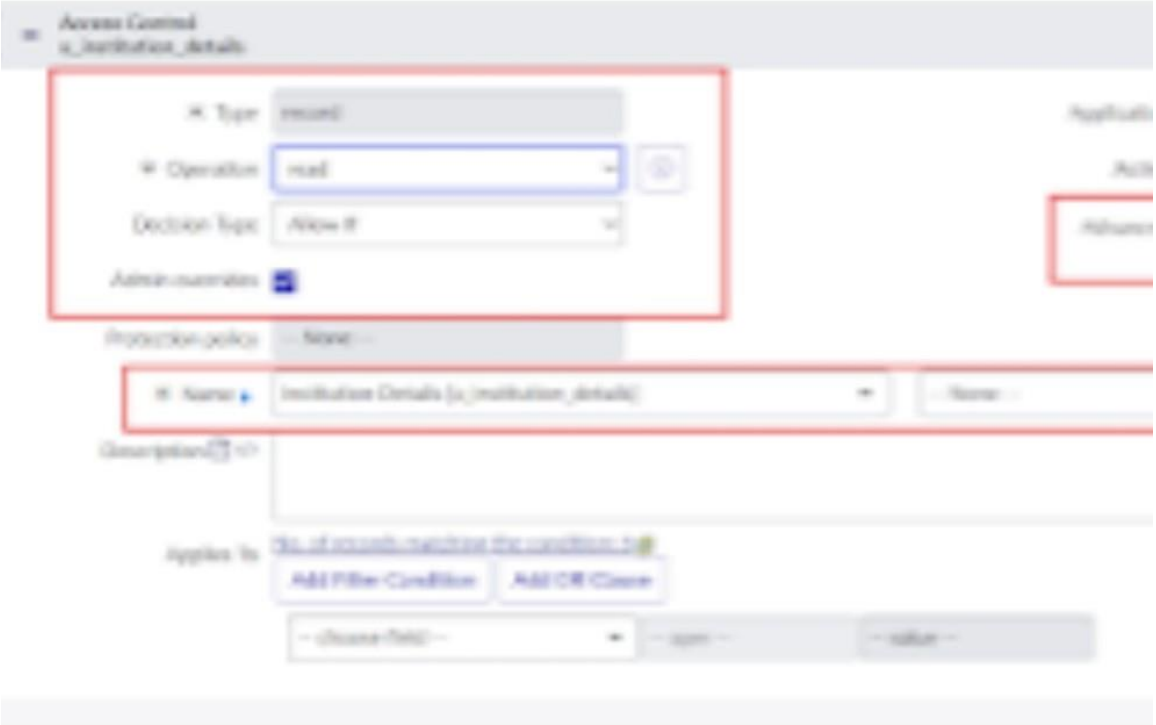
No additional data condition is required here.

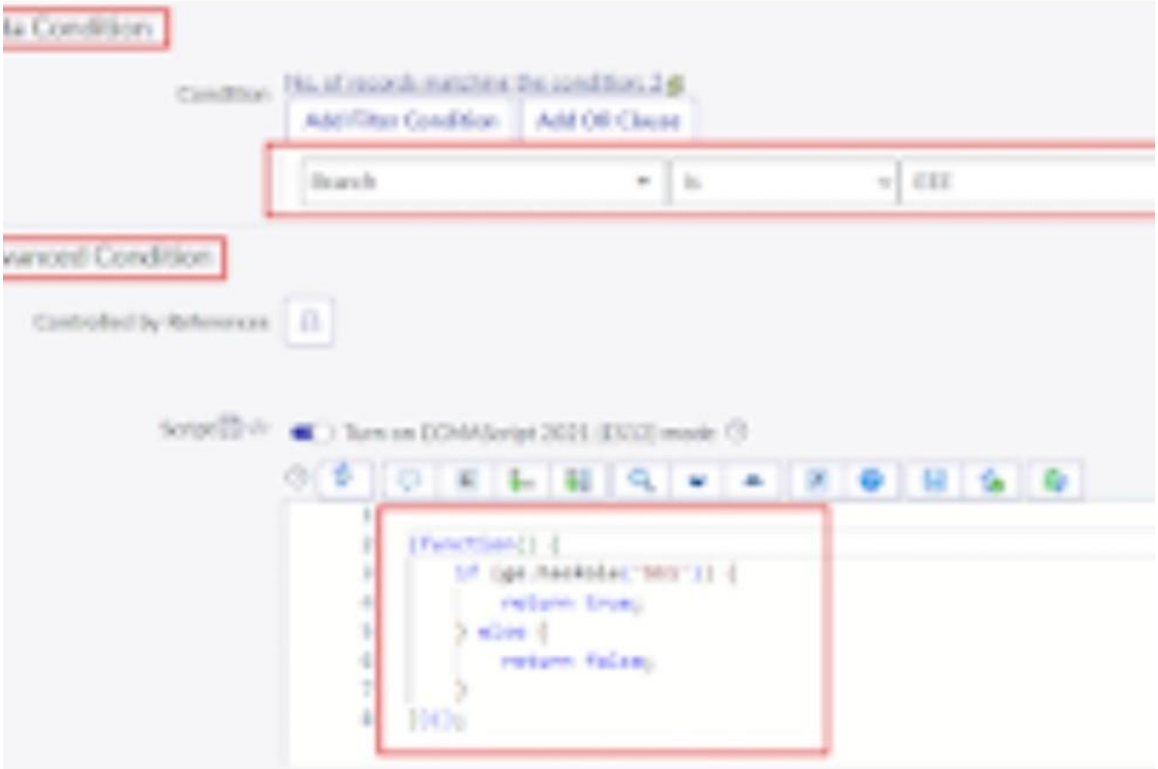
After saving the ACL, I verified the permission.

The user with the bb2 role can create a new record according to the ACL configuration.

This prevents unauthorized users from creating records.

Screenshot – STEP 4 – CREATE ACL


Access Control
u_institution_details
Type
Operation
read
Decision Type
Allow If
Admin overrides
Protection policy
-- None --
Name
Institution Details [u_institution_details]
Description
Applies To
Add Filter Condition
Add OR Clause


Condition
Add Filter Condition
Add OR Clause
Branch
is
Controlled by References
Script
Turn on ECMAScript 2021 (ES12) mode
function() {
if (gs.hasRole('MO')) {
return true;
} else {
return false;
}
})()

STEP 5 – WRITE ACL

Next is the Write ACL.

The operation is Write.

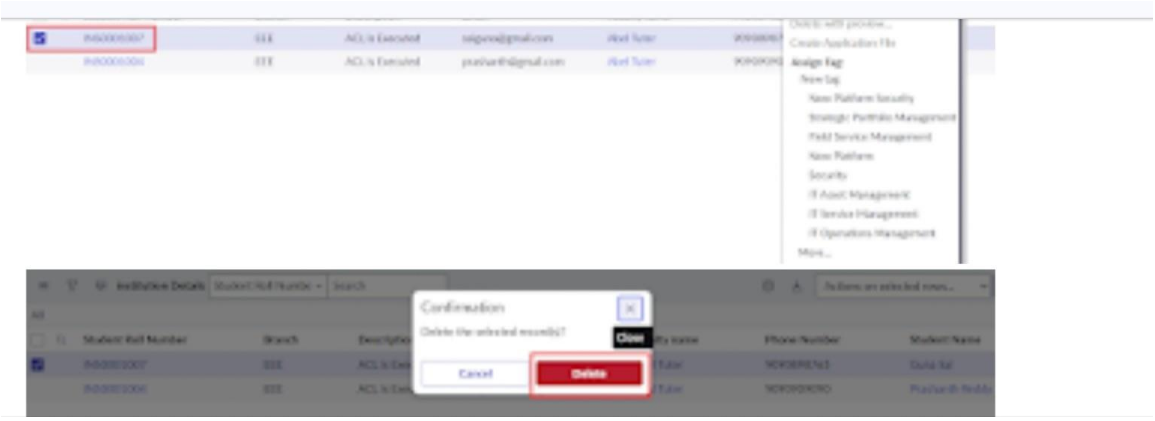
The table is u_institution_details.

For this ACL, I assigned the role bb3.

After saving the ACL, I tested the record modification.

The user with the bb3 role can edit the permitted records. Therefore, the Write ACL controls who can modify existing records.

Screenshot – STEP 5 – WRITE ACL



STEP 6 – DELETE ACL

Finally, I created the Delete ACL.

The operation is Delete.

The table is u_institution_details.

For this ACL, I assigned the role bb4.

After saving the ACL, I tested the deletion permission.

The user with the bb4 role can delete the permitted records.

Other users who do not have the required permission cannot delete the records.

STEP 7 – FINAL VERIFICATION

Now I verified all four operations.

- The bb1 role controls Read access.
- The bb2 role controls Create access.

- The bb3 role controls Write or Edit access.
- The bb4 role controls Delete access.
- The admin user has full access to the records.

Therefore, different users can have different permissions based on their roles.

FIELD-BASED READ SECURITY

The important point is that the Read ACL also checks the Branch field.

So, a user with the bb1 role can see the required EEE records, while other branch records are restricted.

ACCESS CONTROL SUMMARY

CONCLUSION / OUTCOME

To conclude, this project demonstrates how Script-Controlled Access Control Lists can be used in ServiceNow to protect sensitive records.

We created users, roles and a custom table. Then we implemented separate ACLs for Read, Create, Write and Delete operations.

The Read ACL additionally checks the Branch field, providing field-based record access control.

Role	Operation	Permission
bb1	Read	View permitted EEE records
bb2	Create	Create a new record
bb3	Write	Edit permitted records
bb4	Delete	Delete permitted records
Admin	All	Full access